Universidad ICESI

Pruebas End-to-End (E2E) con Playwright

Cúspide de la Pirámide de Pruebas, Auto-Waiting, POM y Validación en CI/CD

Prof. Alejandro Muñoz Bravo

Ingeniería de Software V • Universidad ICESI — Período 2026-2

Ruta de Aprendizaje de la Sesión

1. **La Pirámide de Pruebas y el Rol del E2E**
2. Arquitectura y Ventajas de Playwright
3. Locators Resilientes y Page Object Model (POM)
4. Network Mocking, Diagnóstico y CI/CD Headless

Universidad icesi

01

La Pirámide de Pruebas

De la verificación unitaria a la experiencia real del usuario

¿Por qué las pruebas unitarias no son suficientes?

En sistemas web desacoplados (React + Spring Boot):

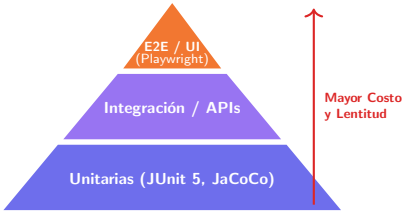
- **Éxito aislado:** Las pruebas unitarias validan lógica de clases y métodos aislados mediante mocks.
- **Punto ciego de integración:** No prueban serialización JSON, cookies ni compatibilidad de navegador.
- **Regresiones en UI:** No detectan si un botón quedó oculto por CSS o si el DOM cambió.
- **Costo de escape:** Los fallos descubiertos en producción por usuarios tienen el mayor costo de resolución.

El Valor del E2E

Las pruebas **End-to-End (E2E)** validan el sistema completo exactamente como lo experimenta el usuario final, desde el clic en pantalla hasta la base de datos.

La Pirámide de Automatización (Cohn / Fowler)

Capas de la Pirámide de Pruebas



Distribución y Compromiso

1. Unitarias (70 %): Rápidas (ms), deterministas, baratas. Cobertura con JaCoCo.

2. Integración (20 %): Contratos REST, base de datos en contenedores.

3. E2E (10 %): Flujos críticos de negocio (Happy paths y recuperación de errores).

Peligro: El antipatrón del "Cono de Helado" genera suites lentas e inestables.

Ruta de Aprendizaje de la Sesión

1. La Pirámide de Pruebas y el Rol del E2E
2. **Arquitectura y Ventajas de Playwright**
3. Locators Resilientes y Page Object Model (POM)
4. Network Mocking, Diagnóstico y CI/CD Headless

Universidad icesi

02

Arquitectura de Playwright

De Selenium WebDriver a la conexión nativa por WebSocket

Evolución: Selenium WebDriver vs Playwright

Selenium (Paradigma Tradicional)

- **Protocolo HTTP REST:** Cada comando realiza una petición HTTP al driver.
- **Polling ineficiente:** Comprobaciones repetitivas de estado en la red.
- **Flakiness alto:** Sensible a tiempos de carga lentos; proliferación de `sleep()`.
- **Aislamiento costoso:** Abrir nuevos navegadores consume alta memoria RAM.

Playwright (Paradigma Moderno)

- **Conexión WebSocket única:** Comunicación bidireccional continua con el motor.
- **Multiplexación de eventos:** Notificación instantánea de cambios en el DOM.
- **BrowserContexts ultraligeros:** Aislamiento total de cookies/cache en ms.
- **Multi-motor real:** Soporte Chromium, Firefox y WebKit en Linux/CI.

Auto-Waiting: Eliminando las Pruebas Intermitentes

Playwright implementa **Actionability Checks** automáticos antes de cada acción:

- **Attached:** El elemento existe en el árbol del DOM.
- **Visible:** No tiene `display:none` ni opacidad 0.
- **Stable:** Ya terminó las animaciones CSS de renderizado.
- **Receives Events:** No está bloqueado por overlays o loaders.
- **Enabled:** El control no se encuentra en estado `disabled`.

Antipatrón Prohibido

Nunca usar pausas fijas: Usar temporizadores ciegos introduce lentitud e intermitencia. Playwright espera activamente resolviendo en el instante exacto en que la condición de accionabilidad se satisface.

Ruta de Aprendizaje de la Sesión

1. La Pirámide de Pruebas y el Rol del E2E
2. Arquitectura y Ventajas de Playwright
3. **Locators Resilientes y Page Object Model (POM)**
4. Network Mocking, Diagnóstico y CI/CD Headless

Universidad icesi

03

Locators Resilientes y POM

Diseño mantenible y selectores basados en accesibilidad W3C/ARIA

Selectores basados en Accesibilidad (W3C / ARIA)

Jerarquía de Locators Recomendada

1. Por Rol (`getByRole`):
Semántica accesible: `button`, `heading`, `dialog`, `textbox`.

2. Por Etiqueta (`getByLabel`):
Asocia el input a su etiqueta `<label>`.

3. Por Texto (`getByText`):
Contenido visible al usuario.

4. Test ID (`getByTestId`):
Atributo `data-testid` cuando el texto varía.

Ejemplo Práctico en TypeScript

```
// Clic resiliente por rol semantico
await page.getByRole('button', {
  name: 'Guardar Producto'
}).click();

// Asercion web-first auto-reintentable
await expect(page.getByRole('alert'))
  .toHaveText('Guardado exitoso');

// Campo por label accesible
await page.getByLabel('Nombre:')
  .fill('Portatil Gamer');
```

El Patrón de Diseño Page Object Model (POM)

¿Qué problema resuelve?

- **Acoplamiento excesivo:** Si un botón cambia en la UI, decenas de archivos de prueba fallan al tiempo.
- **Duplicación de código:** Repetición de secuencias de login o llenado de formularios.
- **Falta de legibilidad:** Pruebas saturadas de selectores HTML en lugar de lógica de negocio.

Estructura y Reglas POM

- **Clase por Página/Componente:** Encapsula los locators y métodos de interacción (ej. `ProductPage.ts`).
- **Separación estricta:** El Page Object interactúa con el DOM; el archivo de prueba (`*.spec.ts`) contiene las aserciones (`expect`).
- **Mantenibilidad:** Un cambio de UI solo requiere actualizar la clase POM en un único lugar.

Estructura Canónica de un Page Object Model (POM)

Page Object: `ProductPage.ts`

```
export class ProductPage {
  readonly page: Page;
  readonly nameInput: Locator;
  readonly saveBtn: Locator;

  constructor(page: Page) {
    this.page = page;
    this.nameInput = page.getByLabel('Nombre:');
    this.saveBtn = page.getByRole('button',
      { name: 'Guardar' });
  }

  async createProduct(name: string) {
    await this.nameInput.fill(name);
    await this.saveBtn.click();
  }
}
```

Archivo de Prueba: `products.spec.ts`

```
test('debe registrar un producto',
  async ({ page }) => {
  const productPage = new ProductPage(page);
  await page.goto('/products');

  await productPage.createProduct('Teclado');

  await expect(page.getByRole('cell',
    { name: 'Teclado' })).toBeVisible();
});
```

Ruta de Aprendizaje de la Sesión

1. La Pirámide de Pruebas y el Rol del E2E
2. Arquitectura y Ventajas de Playwright
3. Locators Resilientes y Page Object Model (POM)
4. **Network Mocking, Diagnóstico y CI/CD Headless**

Universidad icesi

04

Network Mocking y Diagnóstico

Control total de la capa de red y depuración con Trace Viewer

Intercepción de Red con `page.route()`

Escenarios de Prueba de Red

1. Inyección de fallos HTTP:
Simular respuestas 500 Internal Server Error o 401 Unauthorized sin apagar el backend.

2. Simulación de latencia:
Añadir retrasos artificiales para validar spinners y estados de carga.

3. Aislamiento de terceros:
Mockear pasarelas de pago o servicios externos.

Simulación de Fallo de Servidor

```
test('maneja error 500 del servidor',
  async ({ page }) => {
  // Interceptar la API REST
  await page.route('**/api/v1/products',
    async route => {
      await route.fulfill({
        status: 500,
        contentType: 'application/json',
        body: JSON.stringify({
          error: 'Fallo de conexion'
        })
      });
  });

  await page.goto('/products');
  await expect(page.getByRole('alert'))
    .toHaveText('Error al cargar datos');
});
```

Diagnóstico Forense: Playwright Trace Viewer

Capacidades de Inspección Post-Fallo

1. Grabación paso a paso:
Registra cada micro-evento del DOM, captura de pantalla y llamada de red.

2. Explorador temporal:
Permite retroceder en el tiempo y colocar el cursor sobre cada segundo de la prueba.

3. Logs de accionabilidad:
Explica por qué un clic falló (ej. "el elemento tardó más del timeout en ser visible").

Comandos Esenciales de Diagnóstico

```
# Ejecutar con trazas al reintentar:
npx playwright test --trace on-first-retry

# Abrir el visor interactivo de trazas:
npx playwright show-trace trace.zip

# Inspeccionar reporte HTML completo:
npx playwright show-report
```

Integración en Pipelines de CI/CD (GitHub Actions / Jenkins)

Ejecución Desatendida (Headless)

- Se ejecuta sin entorno gráfico (`HEADLESS=true`).
- Consume contenedores o runners efímeros con los navegadores instalados vía `npx playwright install --with-deps`.
- Ejecución paralela masiva por cores o nodos (*Sharding*).

Políticas y Artefactos en CI

- **Retries controlados:** Permite 1 reintento para aislar fallos intermitentes de red.
- **Publicación de artefactos:** Almacena el reporte HTML y los videos/trazas únicamente si el job falla.
- **Quality Gate final:** Si la suite E2E falla, el pipeline bloquea la promoción a producción.

Preguntas de Reflexión y Discusión Profesional

Gobernanza y Estrategia

1. ¿Cuándo pasar de Unitarias a E2E?
Si un equipo tiene poco tiempo, ¿debería priorizar 100 pruebas unitarias o 5 pruebas E2E críticas de checkout?

2. Pruebas Intermitentes (Flaky Tests):
¿Por qué configurar `retries: 3` en CI suele ser un antipatrón que esconde deuda técnica en la arquitectura?

Arquitectura y Mantenibilidad

3. Mantenibilidad con POM:
¿Qué riesgos asume un proyecto grande que escribe selectores directamente en los scripts de prueba sin Page Objects?

4. Network Mocking vs Backend Real:
¿En qué etapa de la entrega continua es preferible mockear el backend y en cuál es obligatorio probar contra base de datos real?